GRAPHS IN C++

A graph is a non-linear data structure that consists of vertices (nodes) connected by edges.

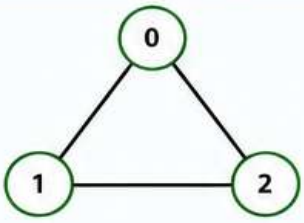
BASIC COMPONENTS

VERTICES (NODES)

The individual objects in a graph.
Represented by circles.

EDGES

The connections between vertices.
Represented by lines.



Vertices: {0, 1, 2}
Edges: {(0-1), (0-2), (1-2)}

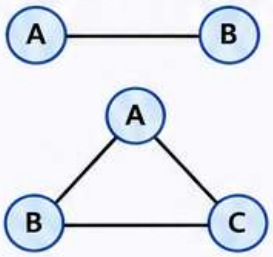
REAL-LIFE APPLICATIONS

- **Google Maps** — Finding shortest routes
- **Social Networks** — Connecting people
- **Flight Routes** — Connections between cities
- **Computer Networks** — Devices connected in a network
- **Recommendation Systems** — Suggesting items to users
- **AI Pathfinding** — Finding optimal paths

TYPES OF GRAPHS

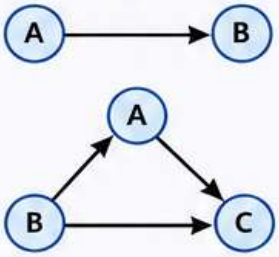
1. UNDIRECTED GRAPH

Edges have no direction.
If A is connected to B, then B is also connected to A.



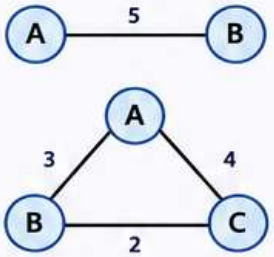
2. DIRECTED GRAPH (DIGRAPH)

Edges have a direction.
If A → B, it does not mean B → A.



3. WEIGHTED GRAPH

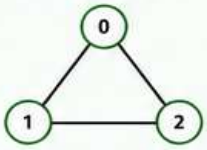
Edges have values (weight) like cost, distance, time, etc.



GRAPH REPRESENTATION IN C++

1. ADJACENCY MATRIX

A 2D array where:
1 → connected exists
0 → no connection



Matrix

	0	1	2
0	0	1	1
1	1	0	1
2	1	1	0

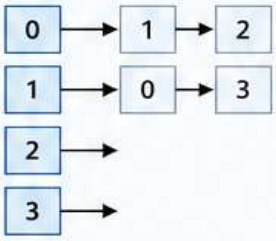
```cpp
#include <iostream>
using namespace std;
int main() {
  int graph[3][3] = {
    {0, 1, 1},
    {1, 0, 1},
    {1, 1, 0}
  };
  for(int i = 0; i < 3;i++) {
    for(int j = 0; j < 3; j++) {
      cout << graph[i][j] << " ";
    }
    cout << endl;
  }
  return 0;
}
```

Output

```
0 1 1
1 0 1
1 1 0
```

2. ADJACENCY LIST (MOST COMMON)

Uses arrays/vectors to store connected nodes.

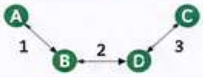


Output

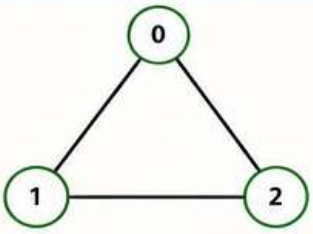
```
Node 0:  1  2
Node 1:  0  3
Node 2:
Node 3:
```

```cpp
#include <iostream>
#include <vector>
using namespace std;
int main() {
  int vertices = 4;
  vector<int> graph[4];
  // Adding edges
  graph[0].push_back(1);
  graph[0].push_back(2);
  graph[1].push_back(0);
  graph[1].push_back(3);
  // Display graph
  for(int i = 0; i < vertices; i++) {
    cout << "Node " << i << ": ";
    for(int neighbor : graph[i]) {
      cout << neighbor << " ";
    }
    cout << endl;
  }
  return 0;
}
```

IMPORTANT GRAPH ALGORITHMS

Algorithm	Purpose	Idea
BFS (Breadth First Search)	Level-wise traversal	
DFS (Depth First Search)	Deep traversal	
Dijkstra (Shortest Path)	Shortest path from source to all nodes	
Prim's Algorithm (Minimum Spanning Tree)	Minimum spanning tree (MST)	
Kruskal's Algorithm (Minimum Spanning Tree)	Minimum spanning tree (MST)	

SIMPLE VISUAL EXAMPLE



Vertices
{0, 1, 2}

Edges
{(0-1), (0-2), (1-2)}

Type
Undirected Graph

This graph shows that every node is connected to the others.

Key Takeaway
Graphs help us model real-world relationships and solve complex problems efficiently using various algorithms.

What this code does

- Creates a graph
- Connects nodes using edges
- Displays all connected nodes

Example Graph

```
0 ---- 1
|     |
|     |
2 ---- 3
```

Connections:

- 0 → 1,2
- 1 → 0,3
- 2 → 0,3
- 3 → 1,2

Easy C++ Code

```
#include <iostream>
#include <vector>

using namespace std;

int main() {

  // Number of vertices
  int vertices = 4;

  // Create graph using adjacency list
  vector<int> graph[4];

  // Add edges
  graph[0].push_back(1);
  graph[0].push_back(2);

  graph[1].push_back(0);
  graph[1].push_back(3);

  graph[2].push_back(0);
  graph[2].push_back(3);

  graph[3].push_back(1);
```

```
    graph[3].push_back(2);

    // Print graph
    for(int i = 0; i < vertices; i++) {

        cout << "Node " << i << " is connected to: ";

        for(int neighbor : graph[i]) {
            cout << neighbor << " ";
        }

        cout << endl;
    }

    return 0;
}
```

Output

Node 0 is connected to: 1 2
Node 1 is connected to: 0 3
Node 2 is connected to: 0 3
Node 3 is connected to: 1 2

Step-by-Step Explanation

1. Create Graph

vector<int> graph[4];

Creates 4 lists:

- graph[0]
- graph[1]
- graph[2]
- graph[3]

Each list stores connected nodes.

---

2. Add Connections

graph[0].push_back(1);

Means:

- Node 0 is connected to Node 1

---

3. Print Connections

for(int neighbor : graph[i])

Loops through all neighbors of a node.

Visual Understanding

graph[0] = 1, 2

graph[1] = 0, 3

graph[2] = 0, 3

graph[3] = 1, 2

Meaning:

0 → 1,2

1 → 0,3

2 → 0,3

3 → 1,2

Why Adjacency List is Better

Advantages:

- Uses less memory
- Faster for large graphs
- Easy to implement

Real-Life Example

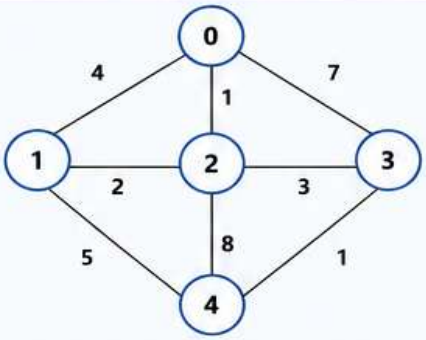
Graph Node	Represents
Person	Social network
City	Maps
Computer	Network
Website	Internet links

Edges represent connections between them.

DIJKSTRA'S ALGORITHM

Dijkstra's Algorithm is used to find the shortest distance from a single source vertex to all other vertices in a weighted graph with **non-negative edge weights**.

EXAMPLE GRAPH



We will find shortest distances from Source = 0

ALGORITHM STEPS

1. **Initialize:**
   - Set distance[source] = 0
   - Set distance[others] = ∞ (infinity)
   - Create a min-priority queue (min-heap) and insert (0, source)
2. Extract the vertex with minimum distance from the priority queue.
3. For each neighbor of the extracted vertex:
   - Calculate new distance = distance[u] + weight(u, v)
   - If new distance is smaller, update distance[v] and push (new distance, v) into the queue.
4. Repeat steps 2–3 until the priority queue becomes empty.
5. The distance array will contain shortest distances from the source to all vertices.

IDEA

Always visit the closest unvisited vertex first (greedy approach).

TIME COMPLEXITY

Using Min-Heap (Priority Queue):

O((V + E) log V)

Where:

V = number of vertices

E = number of edges

C++ IMPLEMENTATION (USING PRIORITY QUEUE)

```cpp
#include <bits/stdc++.h>
using namespace std;

#define INF 1e9

// Dijkstra's Algorithm
void dijkstra(vector<vector<pair<int,int>>> &adj, int src, int V)
{
    vector<int> dist(V, INF);           // Distance array
    dist[src] = 0;                      // Distance to source is 0
    priority_queue<pair<int,int>,
                   vector<pair<int,int>>,
                   greater<pair<int,int>>> pq;   // Min-heap
    pq.push({0, src});                  // {distance, node}

    while(!pq.empty())
    {
        auto top = pq.top(); pq.pop();
        int d = top.first;
        int u = top.second;
        if(d > dist[u]) continue;       // Ignore outdated entry
        for(auto it : adj[u])
        {
            int v = it.first ;          // Neighbor
            int wt = it.second;         // Edge weight
            if(dist[u] + wt < dist[v])  // Relaxation
            {
                dist[v] = dist[u] + wt;
                pq.push({dist[v], v});
            }
        }
    }

    // Print shortest distances
    cout << "Vertex\tDistance from Source "<< src << "\n";
    for(int i=0; i<V; i++)
    {
        cout << i << "\t";
        if(dist[i] == INF) cout << "INF";
        else cout << dist[i];
        cout << "\n";
    }
}

int main()
{
    int V = 5; // Number of vertices

    // Adjacency List: {neighbor, weight}
    vector<vector<pair<int,int>>> adj(V);

    // Add edges
    auto addEdge = [&](int u, int v, int w){
        adj[u].push_back({v, w});
        adj[v].push_back({u, w});
        // Undirected graph
    };


    addEdge(0,1,4);
    addEdge(0,2,1);
    addEdge(0,3,7);
    addEdge(1,2,2);
    addEdge(1,4,5);
    addEdge(2,3,3);
    addEdge(2,4,8);
    addEdge(3,4,1);

    int source = 0;

    dijkstra(adj, source, V);

    return 0;
}
```

EXPLANATION OF CODE

- We use adjacency list to represent the graph.
- Min-heap (priority_queue) always gives the vertex with the smallest distance.
- We relax all neighbors of the current vertex.
- If a shorter path is found, we update the distance and push it into the heap.
- Finally, we print the shortest distance from source to every vertex.

SAMPLE INPUT

Source = 0

(Using the graph shown above)

OUTPUT

Vertex	Distance from Source 0
0	0
1	3
2	1
3	4
4	7

STEP-BY-STEP WORKING (SOURCE = 0)

Step	Extracted Vertex	Queue Content {distance, vertex}	Relaxation / Update	Distance Array [0, 1, 2, 3, 4]				
				0	1	2	3	4
0	–	{0, 0}	Initial	0	∞	∞	∞	∞
1	0	{1,2}, {4,1}, {7,3}	Update dist[2]=1, dist[1]=4, dist[3]=7	0	4	1	7	∞
2	2	{3,1}, {4,3}, {9,4}	Update dist[1]=3, dist[3]=4, dist[4]=9	0	3	1	4	9
3	1	{4, 3}, {8, 4}	Update dist[4]=8	0	3	1	4	8
4	3	{5, 4}	No shorter update	0	3	1	4	8
5	4	{}	Done	0	3	1	4	7

KEY TAKEAWAYS

- Works only with non-negative edge weights.
- Greedy algorithm.
- Finds shortest path from one source to all vertices.
- Uses Priority Queue (Min-Heap) for efficiency.
- Time Complexity: O((V + E) log V)

Dijkstra's Algorithm is widely used in GPS navigation, network routing, and many real-world applications where we need the shortest path from one point to all other points.